

# Assignment 03 – Minimum Spanning Tree

---

## Objective

Implement and compare Kruskal's and Prim's Minimum Spanning Tree algorithms.

Both algorithms use the same weighted, undirected graph and the graph is converted from adjacency list format to CSR before the algorithm runs.

## Algorithm / Approach

- Kruskal's algorithm sorts all edges by weight and adds an edge if it does not create a cycle.
- A DSU (Union-Find) structure is used for cycle detection in Kruskal.
- Prim's algorithm starts from vertex 0 and repeatedly selects the cheapest edge connecting the current tree to an unvisited vertex.
- The adjacency-list input is converted to CSR before either algorithm runs.
- CSR conversion and file reading are not included in the reported execution time.
- Both algorithms are run on the same graph files.
- The total MST weight produced by both algorithms must be the same.

## Input Format

The input contains:

V E

Then one line for every vertex:

vertex degree neighbour weight neighbour weight ...

The graph is weighted and undirected. Each edge appears in both endpoint adjacency lists, while E counts each undirected edge once.

## Input Example

```
5 7
0 2 1 2 3 6
```

```
1 4 0 2 2 3 3 8 4 5
2 2 1 3 4 7
3 3 0 6 1 8 4 9
4 3 1 5 2 7 3 9
```

## Output Example

```
Algorithm: Kruskal's MST
MST edges:
0 1 2
1 2 3
1 4 5
0 3 6
Total MST weight: 16
Execution time: <value> ms
```

```
Algorithm: Prim's MST
MST edges:
0 1 2
1 2 3
1 4 5
0 3 6
Total MST weight: 16
Execution time: <value> ms
```

## Compilation and Execution

Compile the project from the `assignment_03` folder:

```
g++ driver/.cpp src/.cpp -o assignment_03
```

Run:

```
./assignment_03
```

The program allows Kruskal, Prim, or both algorithms to be selected.

## Test Cases and Results

The large graphs were generated as connected sparse graphs with approximately  $E = 2V$ .

---

| Test File  | Vertices | Edges   | Kruskal Weight | Prim Weight | Kruskal Time | Prim Time  | Status |
|------------|----------|---------|----------------|-------------|--------------|------------|--------|
| mst_10     | 10       | 19      | 275            | 275         | 0.0116 ms    | 0.0146 ms  | Pass   |
| mst_100    | 100      | 199     | 2727           | 2727        | 0.0829 ms    | 1.2122 ms  | Pass   |
| mst_10000  | 10,000   | 19,999  | 279085         | 279085      | 248.605 ms   | 8023.45 ms | Pass   |
| mst_50000  | 50,000   | 99,999  | 1620317        | 1620317     | 4545.23 ms   | 263819 ms  | Pass   |
| mst_100000 | 100,000  | 199,999 | 3894294        | 3894294     | 9693.22 ms   | 883702 ms  | Pass   |

The MST weights match for all test cases.

## Complexity

- Kruskal:  $O(E \log E)$
- Prim (current simple implementation):  $O(V \times E)$
- CSR conversion:  $O(V + E)$
- Kruskal space:  $O(V + E)$
- Prim space:  $O(V)$

## Time Complexity

- Kruskal:  $O(E \log E)$

Kruskal first sorts all  $E$  edges by weight. Sorting is the main operation, so the overall complexity is  $O(E \log E)$ .

- Prim:  $O(V \times E)$

In our implementation, Prim searches through the graph to find the cheapest edge and repeats this for each of the  $V$  vertices. This gives  $O(V \times E)$ .

- CSR conversion:  $O(V + E)$

The graph is converted to CSR by going through the vertices and edges once. This is preprocessing and is not included in the reported algorithm time.

## Output Files

The output files contain the selected MST edges, total MST weight, and execution time for both algorithms.

- mst\_10\_output.txt
- mst\_100\_output.txt
- mst\_10000\_output.txt
- mst\_50000\_output.txt
- mst\_100000\_output.txt

## References

CS509 Assignment 3 Guidelines